LensLab Studio

Selenium Test Suite — Technical Documentation

Property	Value
Document ID	st2420018_py_doc
Language	Python 3.12+
Framework	pytest 8+
Automation Tool	Selenium WebDriver 4+
Target Browser	Google Chrome (mandatory)
Target Application	LensLab Studio (index.html)
Test Credentials	username: admin / password: @Dm1n

1. Project Overview

LensLab Studio is a single-page web application for camera and lens equipment management. The Selenium test suite provides automated end-to-end (E2E) coverage of all critical user flows — from authentication through multi-page navigation and contact form validation to logout. Tests are written in Python 3 using pytest as the test runner and Selenium WebDriver for browser automation.

1.1 Goals

- Verify that the login flow correctly authenticates valid users and rejects invalid credentials.
- Confirm that navigation between the Cameras, Lenses, and Contact pages works correctly.
- Validate all contact form fields — required checks, format validation, and real-time feedback.
- Ensure the logout button returns the application to the unauthenticated state.
- Serve as a regression safety net: any future change that breaks existing behaviour is caught immediately.

2. Technology Stack

Technology	Version	Role
Python	3.12+	Core programming language
pytest	8+	Test runner, fixture management, reporting

Selenium WebDriver	4+	Browser automation — simulates real user interaction
Google Chrome	latest	Target browser (mandatory per assignment)
WebDriver Manager	4+	Automatic ChromeDriver installation and updates

3. Architecture — Page Object Model

The test suite is built around the Page Object Model (POM) design pattern. POM separates the test logic from the UI interaction details, making the code maintainable, readable, and resilient to UI changes.

3.1 Layer Overview

Layer	Class / Module	Responsibility
Selector Registry	Selectors	Centralised store of all element locators (By.ID, By.XPATH, etc.). Change a selector in one place and all tests are updated automatically.
Base Layer	BasePage	Generic Selenium operations: find_element (with explicit wait), click, type_text, is_displayed, execute_js. All page objects inherit from this class.
Page Objects	LoginPage	High-level methods for login actions: login(credentials), get_error_message(), get_user_initials().
	AppPage	High-level methods for the main application: navigate_to_page(page), is_page_visible(page), logout().
	ContactFormPage	High-level methods for the contact form: fill_contact_form(), submit_form(), get_field_error(), get_success_message().
Fixtures	conftest-level	pytest fixtures that create and inject page objects, provide valid credentials and test data, and manage the WebDriver lifecycle.
Test Classes	TestLogin, etc.	Test methods that use fixtures and page objects. No raw Selenium code — only high-level domain calls.

3.2 Key Design Decisions

Selectors Registry

All CSS selectors and XPath expressions are stored in the Selectors class. When the application HTML changes, only this single file needs to be updated — no hunting through dozens of test methods.

WebDriverWait over time.sleep()

Explicit waits (WebDriverWait + expected_conditions) are used instead of fixed time.sleep() pauses. This makes tests faster on fast machines and more reliable on slow CI servers. The remaining time.sleep(0.5) calls are acknowledged as areas for improvement (see Section 6).

ensure_logged_in() Helper

Rather than repeating login steps in every test, the ensure_logged_in() helper checks whether the application is already visible before attempting a new login. This prevents unnecessary round-trips while keeping each test class independent.

4. Test Structure — AAA Pattern

Every test in the suite follows the Arrange–Act–Assert (AAA) pattern, also described in the documentation as Setup–Action–Assertion:

Phase	Description	Example
Setup (Arrange)	Load the page, authenticate if needed, navigate to the target section, clear any stale state.	ensure_logged_in(); app_page.navigate_to_page(PageName.CONTACT)
Action (Act)	Perform the user interaction being tested.	contact_form_page.submit_form()
Assertion (Assert)	Inspect the DOM and verify the outcome matches the expectation.	assert error == 'Name is required.'

5. Test Cases

5.1 TestLogin

Test Method	Action	Expected Result
test_login_page_visible_on_startup	Open index.html	#loginPage is visible
test_invalid_credentials_error	Enter wrong username/password, click Login	Error: 'Incorrect username or password.'
test_valid_login_shows_app	Enter admin/@Dm1n, click Login	#app is visible; #loginPage is hidden
test_user_initials_displayed	Log in with valid credentials	#userBadge shows 'AD'

5.2 TestNavigation

Test Method	Action	Expected Result
test_cameras_visible_by_default	Log in	Cameras section visible immediately
test_navigate_to_lenses	Click Lenses nav btn	Lenses section visible
test_navigate_to_contact	Click Contact nav btn	Contact section visible
test_navigate_back_to_cameras	Click Cameras nav btn	Cameras section visible again
test_only_one_page_visible	Navigate to Lenses	Cameras and Contact are hidden

5.3 TestContactFormValidation

Test Method	Action	Expected Result

test_name_required	Submit empty form	'Name is required.'
test_email_required_when_empty	Type then delete email	'Email address is required.'
test_email_invalid_format	Type 'not_an_email' in email field	Error contains 'valid email'
test_email_valid_clears_error	Type valid email	Email error is empty string
test_phone_required_when_empty	Type then delete phone	'Phone number is required.'
test_phone_too_short	Type '123' in phone field	Error contains '7–15 digits'
test_phone_valid_clears_error	Type valid phone number	Phone error is empty string

5.4 TestContactFormSubmission & TestLogout

Test Method	Action	Expected Result
test_successful_submission	Fill all fields, click Submit	#formSuccess: 'Message sent successfully!'
test_logout_returns_to_login	Click logout button	#loginPage becomes visible; #app hidden

6. Installation & Execution (Windows)

Step 1 — Install Python

Download Python 3.12+ from python.org. During installation, tick the option "Add Python to PATH".

Step 2 — Create a Virtual Environment

```
python -m venv venv
```

Step 3 — Activate the Environment

```
.\venv\Scripts\activate
```

If PowerShell blocks the activation script, run this once (requires administrator):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Step 4 — Install Dependencies

```
pip install selenium pytest webdriver-manager
```

Step 5 — Run the Tests

```
pytest st2420018_py_test.py -v
```

7. Submitted Files

Filename	Description
st2420018_py_test.py	The main Python/pytest/Selenium test file (hosted on GitHub)
st2420018_py_doc.pdf	This technical documentation document
st2420018_py_present.pdf	Presentation slides (6 slides) for the oral defence